

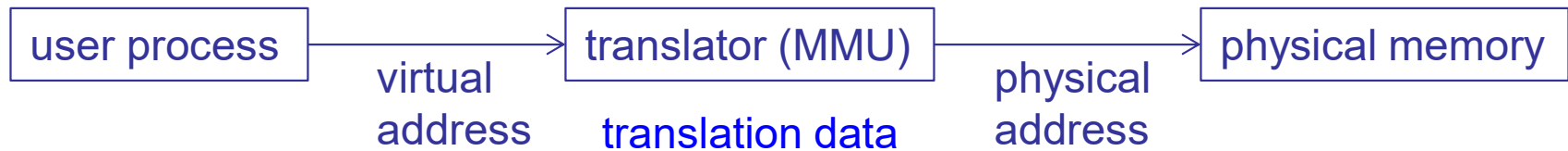
EECS 482

Introduction to operating systems

User and kernel address spaces

Peter Chen
University of Michigan

Dynamic address translation

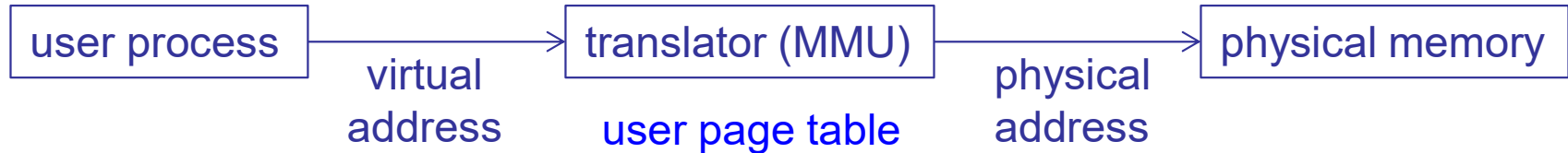


- Many algorithms and data structures can be used to implement translator
 - Base and bound
 - Segmentation
 - Paging
 - Multi-level paging

Where to store page tables?

- In physical or virtual memory?
 - I.e., does PTBR, etc. contain a physical or virtual address?
- Option 1: store page tables in physical memory
- Option 2: store page tables in virtual memory
 - Benefits?
 - Problems?
 - Solution?
 - Project 3: page tables for user processes are stored in kernel's address space. Kernel's address space is managed by infrastructure.

User and kernel address spaces



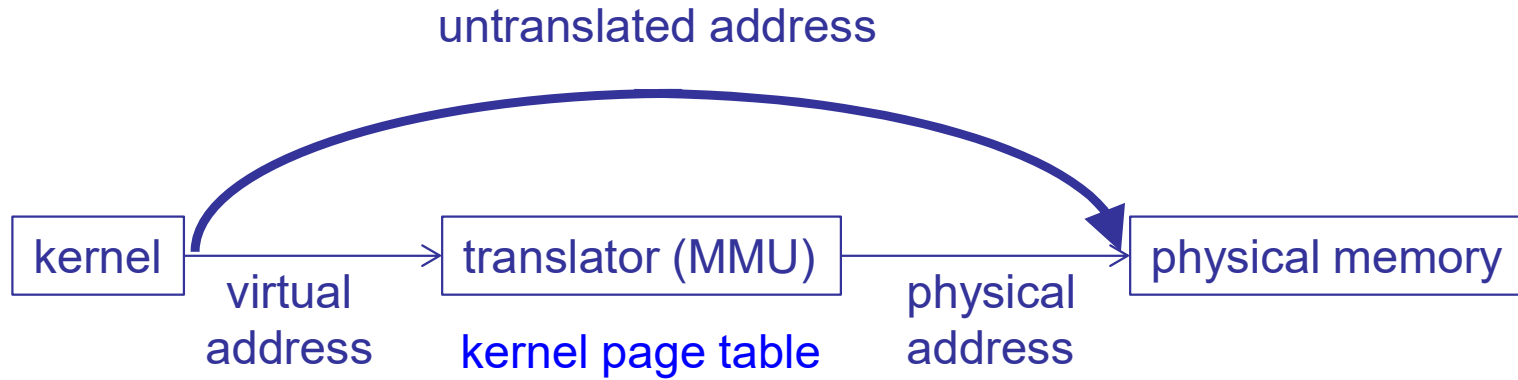
- Each process has its own address space
 - Kernel sets up page table per process
 - MMU looks up current page table (pointed to by PTBR) to translate a virtual address to a physical memory address (or trap to OS)
- What about kernel's address space? Are kernel loads/stores translated?
 - Kernel is like a privileged process, with its own address space
 - Project 2 kernel?
 - Project 3 kernel?

Kernel vs. user address spaces

- Kernel address space is similar to a user address space, with a few key differences
 1. Can you evict the kernel's virtual pages?
 - Some pages are “pinned” to physical memory
 2. Sometimes the kernel must access specific physical memory addresses

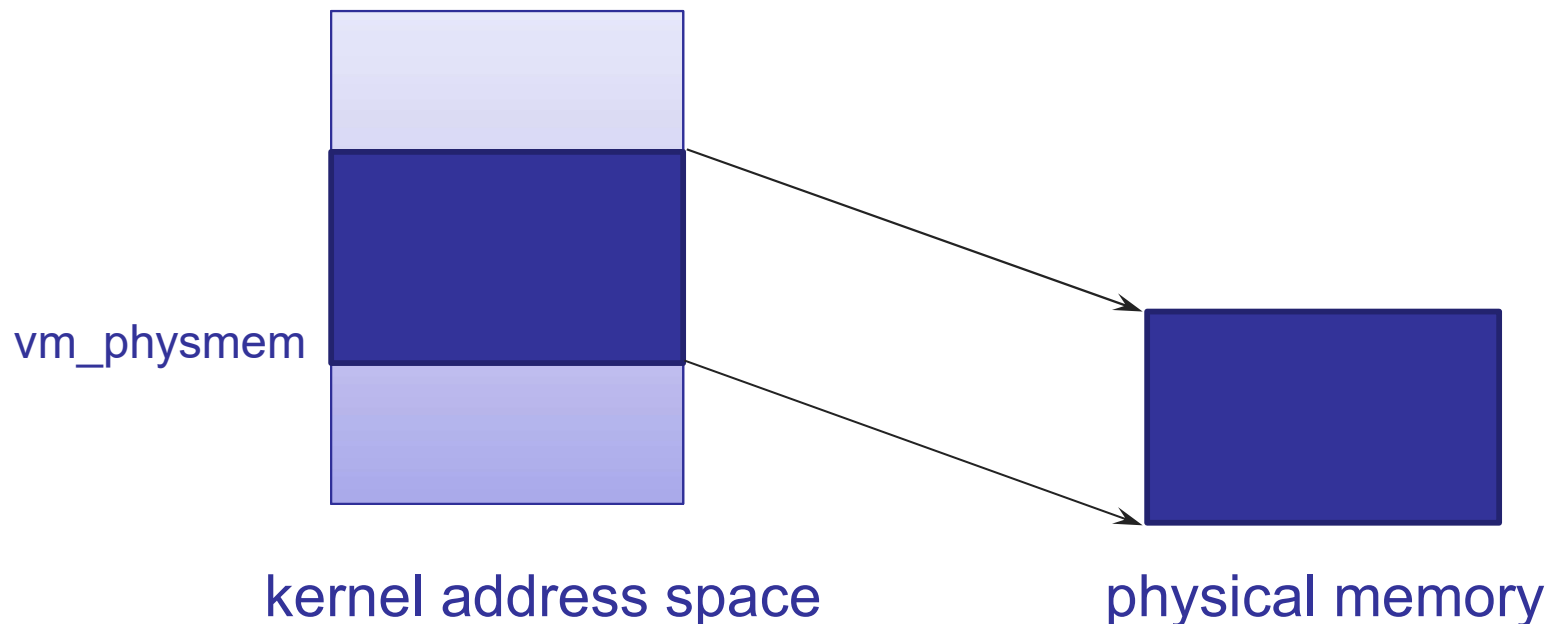
How does kernel access specific physical memory addresses?

- Option 1: Issue an untranslated address (i.e., bypass the MMU)



How does kernel access specific physical memory addresses?

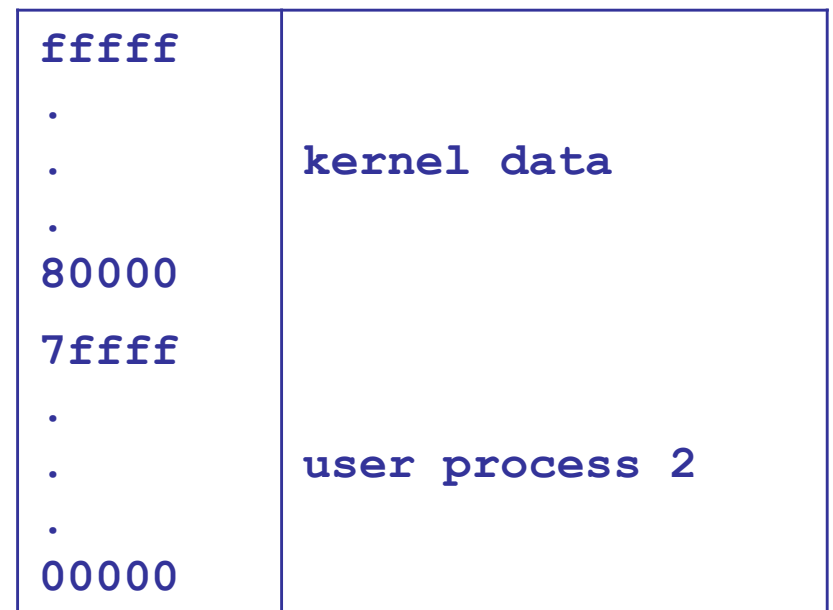
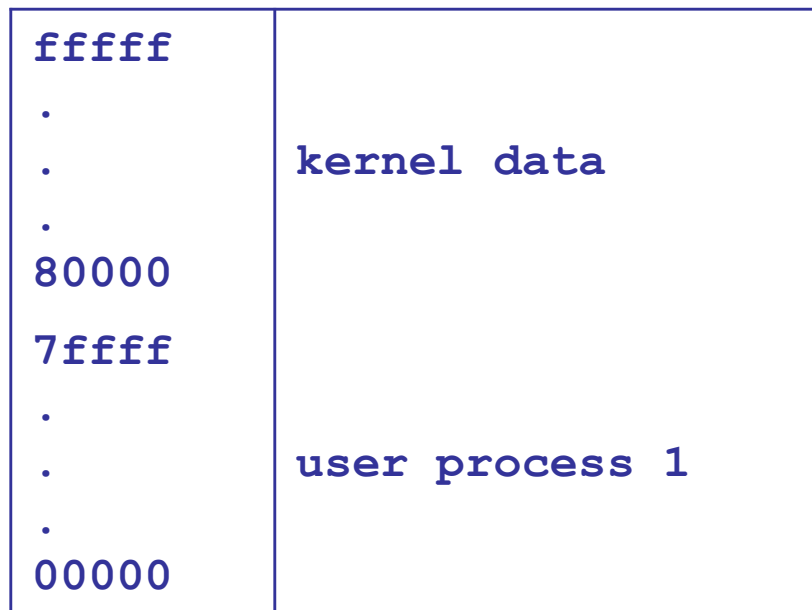
- Option 2: Create a mapping from virtual address range to physical memory range
 - Project 3: `vm_physmem[n]` maps to byte `n` of physical memory
 - **Problems?**



Kernel vs. user address spaces

3. Sometimes the kernel must access data in a **different** address space

- Method A: Find the data in physical memory or disk
- Method B: Map kernel address space into every process's address space, then issue virtual address



Protection: kernel/user mode

- How are we protecting a process's address space from other processes?
- Who is allowed to modify the translation data?
- How does CPU know kernel is running?
 - Hardware support: mode bit
- Who is allowed to change the mode bit?
 - Kernel?
 - User?
 - Problem?

How can user process safely switch to kernel mode?

- Faults and interrupts
 - Why are these safe to transfer control to kernel?
- System calls
 - Process management: fork()/exec()
 - Memory management: mmap
 - I/O: open(), close(), read(), write()
 - System management: reboot()
 - ...

System calls

- When you call `cout <<` in your C++ program:
 - `cout` calls `write()`, which executes assembly-language instruction `syscall`
 - `syscall` traps to kernel at **pre-specified** location
 - kernel's `syscall` handler calls kernel's `write()`
- To handle trap to kernel, hardware atomically
 - Sets mode bit to kernel
 - Saves registers, PC, SP
 - Changes SP to kernel stack
 - Changes to kernel's address space
 - Jumps to exception handler
 - **What does this remind you of?**

Passing arguments to system calls

- Which address space (user process's or kernel's) should be used to store the arguments?
- Kernel must carefully check validity of arguments
 - E.g., `write(int fd, void* buf, size_t size)`
 - » Is `fd` a valid descriptor for an open file?
 - » Are all addresses in `[buf, buf+size)` valid?
 - » Are all addresses in `[buf, buf+size)` readable?

Address space protection: summary

- How are address spaces protected?
 - Separation of translation data
- How is translation data protected?
 - Can update translation data only if mode bit set
- How is mode bit protected?
 - Sets/reset mode bit when transitioning from user-level to kernel-level code and back
 - Transitions limited by interrupt vector table
- Protection boils down to init process which sets up interrupt vector table when system boots up